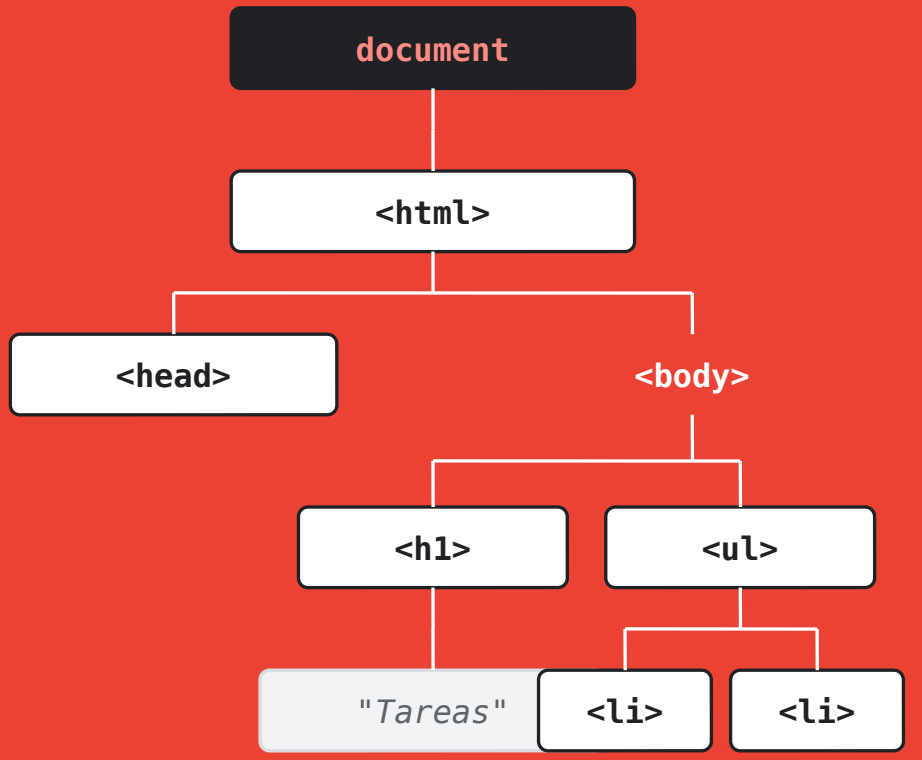
MÓDULO DWEC · DAW — UNIDAD DIDÁCTICA 5

# DOM nativo y gestión de eventos

RA4 · Árbol del documento, manipulación dinámica, eventos y formularios

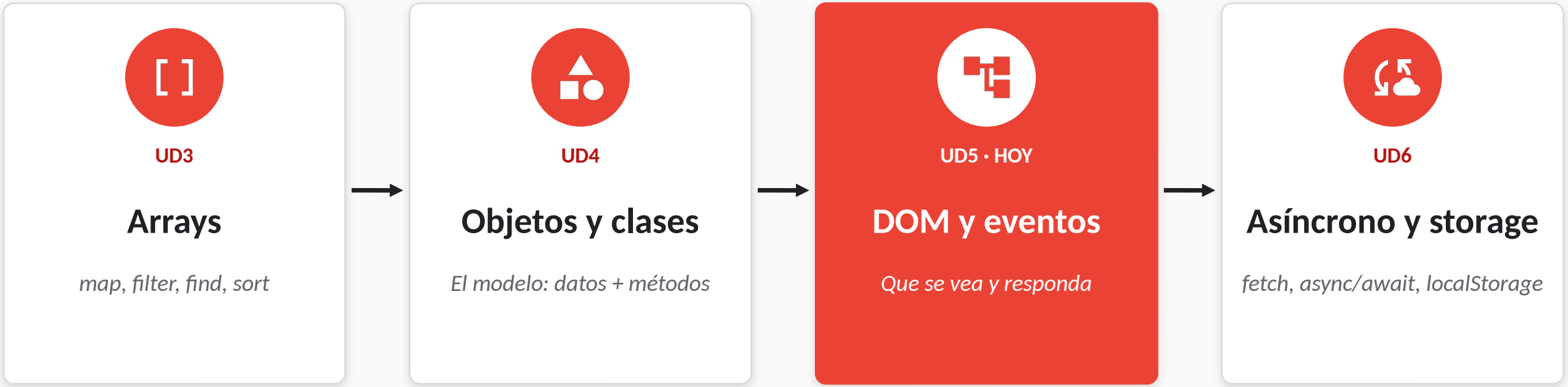
- Entender la página como un árbol de objetos que JavaScript puede cambiar
- Seleccionar, crear y modificar nodos sin recargar y sin abrir agujeros de seguridad
- Responder al usuario con eventos, delegación y formularios validados




*Cada caja es un objeto: se puede leer, cambiar, crear o borrar desde JavaScript.*

DÓNDE ESTAMOS

# Del modelo en memoria a la página que responde




**La pregunta de la unidad:** ¿cómo hago que lo que pasa en mi modelo se vea en la página, y que lo que hace el usuario en la página cambie el modelo?

**OBJETIVO —** La Práctica 2 (tablero de tareas) pinta con el DOM el modelo de la UD4. Calentamiento B: el viernes 06/11, al final de la unidad.

CONTEXTO PROFESIONAL

# ¿Qué pasa cuando pulsas «Añadir al carrito»?

### El ciclo de cualquier interfaz



**1 · Evento**

`el usuario pulsa un botón`

**2 · Modelo**

`carrito.añadir(producto)`

**3 · Render**

`se repinta el contador y la lista`

***…y vuelta a esperar el siguiente evento.***

### Sin recargar la página

Gmail, Trello, Spotify web: todo cambia sin pedir una página nueva

### Debajo de React y Vue

Los frameworks (UD7) hacen esto por ti, pero con el mismo DOM

### Seguridad

Meter datos del usuario con innerHTML es la puerta del XSS

### Accesibilidad

Un div con onclick no se usa con teclado; un button, sí

### Rendimiento

Tocar el DOM es lo más caro que hace tu JavaScript

**DATO CLAVE —** El DOM es un estándar del W3C/WHATWG, no de JavaScript: el mismo árbol se maneja desde cualquier lenguaje que hable con el navegador.

HOJA DE RUTA

# Cinco bloques, del árbol al formulario

1

## El árbol DOM

nodos, document, cuándo está listo

2

## Seleccionar y leer

querySelector, textContent, dataset

3

## Modificar y crear

classList, createElement, template

4

## Eventos

propagación, delegación, this

5

## Formularios

FormData, validación, errores

### Laboratorio

Agenda de contactos con DOM y eventos

### DevTools

Elements, Event Listeners, breakpoints de DOM

### Mini-reto

Pestañas accesibles en parejas

**CLAVE —** Esta unidad se aprende con la consola abierta y DevTools en la pestaña Elements. Lo que no veáis cambiar ahí, no ha pasado.



BLOQUE 1

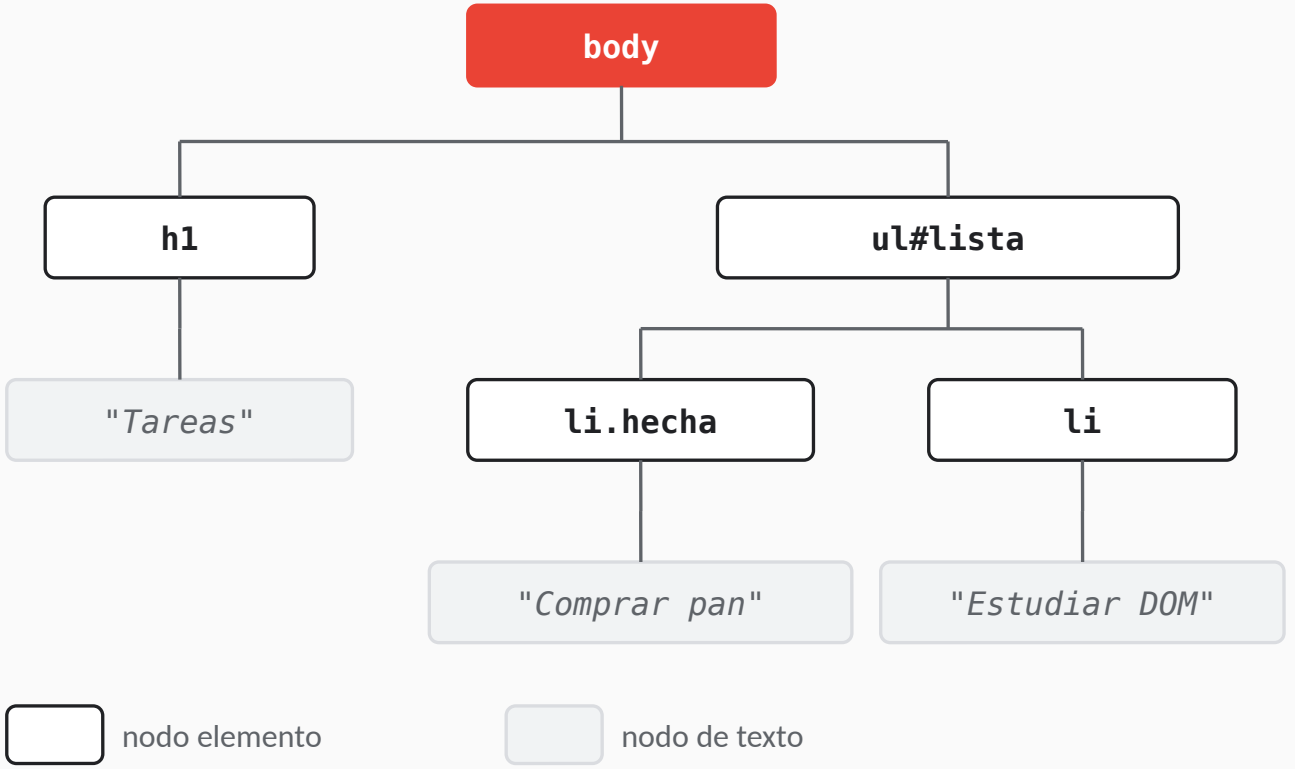
# El árbol DOM

Del HTML a un árbol de objetos: nodos, document, window y cuándo está listo

# El navegador lee el HTML y construye un árbol de objetos

index.html

```html
<body>
  <h1>Tareas</h1>
  <ul id="lista">
    <li class="hecha">Comprar pan</li>
    <li>Estudiar DOM</li>
  </ul>
</body>
```




**Etiquetas → nodos elemento · texto → nodos de texto · anidar → padre e hijos**

**CLAVE —** DOM = Document Object Model: el documento como árbol de OBJETOS. Cambias un objeto y la página cambia al momento.

# No todo lo que hay en el árbol es una etiqueta

| Tipo | nodeType | Ejemplo | Cómo lo ves |
|---|---|---|---|
| **Elemento** | 1 | <li>, <button> | document.querySelector("li") |
| **Texto** | 3 | "Comprar pan", ¡y los saltos de línea! | li.firstChild |
| **Comentario** | 8 | <!-- nota --> | Raro de usar |
| **Documento** | 9 | document | La raíz de todo |

consola

```
const ul = document.querySelector("#lista");
ul.childNodes.length;  // 5  ← texto, li, texto, li, texto
ul.children.length;    // 2  ← solo los <li>
ul.firstChild;         // #text "\n    "
ul.firstElementChild;  // <li class="hecha">
```

### ¿De dónde salen esos nodos de texto?

- Los saltos de línea y espacios entre etiquetas también son texto
- childNodes los incluye; children solo trae elementos
- Usad siempre las versiones Element: children, firstElementChild, nextElementSibling…

**NOTA —** Node es la clase base; Element hereda de Node, y HTMLLIElement de HTMLElement. Otra cadena de prototipos (UD4).

# window es el navegador; document es la página

**window (objeto global del navegador · BOM)**

**document (el DOM)**

<html>

<head> · <body>

*elementos, textos…*

location

history

localStorage

console

setTimeout

navigator

location → URL · history → atrás/adelante · localStorage → UD6

consola

```
window.document === document; // true
window.innerWidth;   // ancho de la ventana
location.href;       // URL actual
document.title = "Hola"; // cambia la pestaña
typeof window;       // "object"
```

**Global:** en un script clásico, una variable con var o una función declarada acaba en window. En los módulos de Vite, no: cada fichero tiene su propio ámbito.

**NOTA —** BOM = Browser Object Model: todo lo que el navegador ofrece además de la página. No es un estándar único, pero hoy todos lo implementan igual.

# Si buscas un elemento antes de que exista, obtienes null




| Cómo cargas el script | ¿Cuándo se ejecuta? | ¿Ve el body? |
|---|---|---|
| **<script src> en <head>** | En cuanto lo encuentra, parando la lectura | ✗ null |
| **<script src> al final de <body>** | Al llegar ahí | ✓ |
| **<script defer src>** | Tras leer todo el HTML, antes de DOMContentLoaded | ✓ |
| **<script type="module" src> (Vite)** | Diferido como defer, siempre | ✓ |

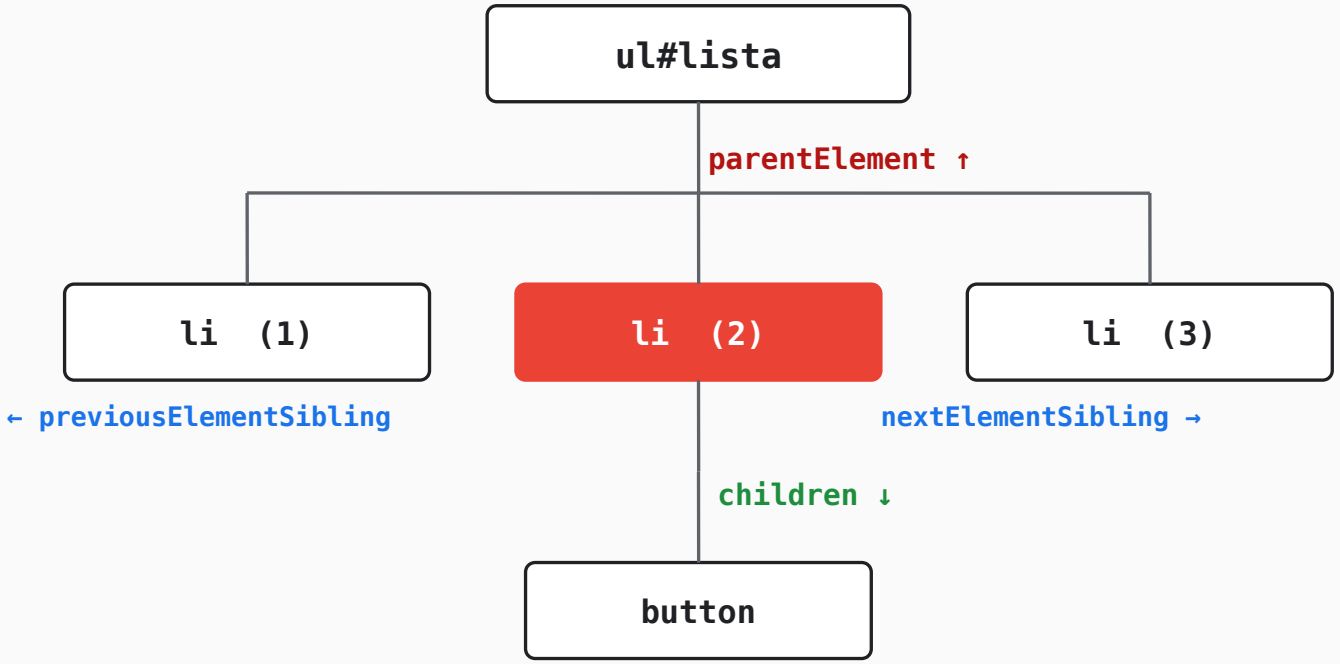
main.js

```
// con type="module" no hace falta,
// pero lo veréis en mucho código:
document.addEventListener(
  "DOMContentLoaded",
  () => iniciar()
);
```

*Vite ya pone type="module" en el index.html: el árbol está listo cuando se ejecuta main.js.*

**CLAVE —** «Cannot read properties of null» al empezar el script = buscaste el elemento antes de que existiera (o el selector está mal).

# Padres, hijos y hermanos




consola

```
const li = lista.children[1];  // el 2.º
li.parentElement;              // <ul id="lista">
li.previousElementSibling; // li (1)
li.nextElementSibling;     // li (3)
li.children;               // [button]
li.firstElementChild;      // <button>

// subir hasta el antecesor que cumpla
const btn = li.querySelector("button");
btn.closest("ul");      // <ul id="lista">
btn.closest(".nada");   // null
```

**closest(selector):** empieza en el propio elemento y sube por los padres. Es la pieza clave de la delegación de eventos (bloque 4).

**NOTA —** Moverse por el árbol con nextSibling y compañía es frágil: un cambio en el HTML lo rompe. Preferid seleccionar con querySelector o subir con closest.



BLOQUE 2

# Seleccionar y leer

querySelector, colecciones, textContent, atributos frente a propiedades y dataset

# Los mismos selectores que en CSS

| Método | Devuelve | Si no hay ninguno |
|---|---|---|
| `querySelector(sel)` | El PRIMERO que cumple | null |
| `querySelectorAll(sel)` | NodeList con todos (estática) | NodeList vacía |
| `getElementById(id)` | El elemento con ese id (sin #) | null |
| `el.querySelector(sel)` | Busca solo DENTRO de el | null |

## Buenas costumbres

- Guarda el elemento en una const al principio: no lo busques en cada clic
- Selecciona por id, clase o data-* propios, no por la estructura (div > div > span)
- Comprueba null si el elemento puede no estar: el?.classList…

seleccionar.js

```
document.querySelector("#lista");
document.querySelector(".hecha");
document.querySelector("ul > li:last-child");
document.querySelector("[data-id='3']");
document.querySelector("input[name=email]");

const items =
  document.querySelectorAll("li");
items.length;
items.forEach(li =>
  console.log(li.textContent));

// dentro de un elemento:
lista.querySelectorAll("li.hecha");
```

**CLAVE —** Si conocéis los selectores de CSS (DIW), ya sabéis seleccionar en el DOM. Es literalmente la misma sintaxis.

# NodeList estática frente a HTMLCollection viva

### querySelectorAll → NodeList estática

Foto del momento: si después añades un li, la lista NO cambia. Tiene forEach. No tiene map ni filter.

### children / getElementsBy... → HTMLCollection viva

Se actualiza sola al cambiar el DOM. No tiene forEach. Recorrerla mientras borras salta elementos.

estatica-viva.js

```
const fija = document.querySelectorAll("li");
const viva = ul.children;
ul.append(document.createElement("li"));
fija.length; // 2  (la foto)
viva.length; // 3  (se ha enterado)
```

a-array.js

```
// ¿quieres map o filter? conviértela
const textos = [...document.querySelectorAll("li")]
  .map(li => li.textContent);
const hechas = Array.from(ul.children)
  .filter(li => li.classList.contains("hecha"));
```

**CLAVE —** [...lista] o Array.from(lista) y ya tenéis todo lo de la UD3: map, filter, find, some...

# textContent, innerText e innerHTML no son lo mismo

tarjeta.html

```
<p id="p">Hola <b>Ana</b>
  <span hidden>oculto</span></p>
```

**Para leer y escribir texto, textContent.** Es rápido, no interpreta HTML y es seguro (bloque 3). innerText solo si necesitas «lo que se ve».

| Propiedad | Devuelve | Ejemplo con #p |
|---|---|---|
| `textContent` | Todo el texto, tal cual, también lo oculto | "Hola Ana\n  oculto" |
| `innerText` | El texto como se VE (respeta CSS, más lento) | "Hola Ana" |
| `innerHTML` | El HTML de dentro, como texto | "Hola Ana\n  <span hidden>…" |

valores de formulario: otra cosa

```
const email = document.querySelector("#email").value;  // lo escrito AHORA (string)
```

**NOTA —** Los <input> no tienen texto dentro: lo que el usuario escribe está en .value. textContent de un input es "".

# El HTML dice cómo EMPIEZA; la propiedad dice cómo ESTÁ

```
<input id="cant" value="1">
```

**El usuario borra el 1 y escribe 5:**

| atributo value | propiedad .value |
|---|---|
| "1" (el HTML original) | "5" (lo que hay ahora) |

consola

```
cant.getAttribute("value"); // "1"
cant.value;                 // "5"
check.checked;              // true / false
enlace.getAttribute("href"); // "/ayuda"
enlace.href;  // URL completa: "http://…/ayuda"
```

| Para... | Usa |
|---|---|
| el estado actual de un control | .value · .checked · .disabled |
| atributos propios o ARIA | setAttribute("aria-expanded", "true") |
| quitar un atributo | removeAttribute("hidden") |
| saber si está | hasAttribute("required") |
| clases y data-* | classList · dataset (siguientes) |

**Booleanos:** hidden, disabled, required, checked valen por existir. setAttribute("disabled", "false") ¡lo DESACTIVA! Usad la propiedad: boton.disabled = false.

**CLAVE —** Atributo = texto del HTML. Propiedad = valor vivo del objeto. Para controles de formulario, casi siempre la propiedad.

# data-* une cada elemento con su dato del modelo

```html
<li data-id="42" data-prioridad="alta" data-fecha-limite="2026-11-06">Entregar</li>
```

| | | | |
|---|---|---|---|
| data-id | → | dataset.id | "42" |
| data-prioridad | → | dataset.prioridad | "alta" |
| data-fecha-limite | → | dataset.fechaLimite | "2026-11-06" |

guiones → camelCase · SIEMPRE string

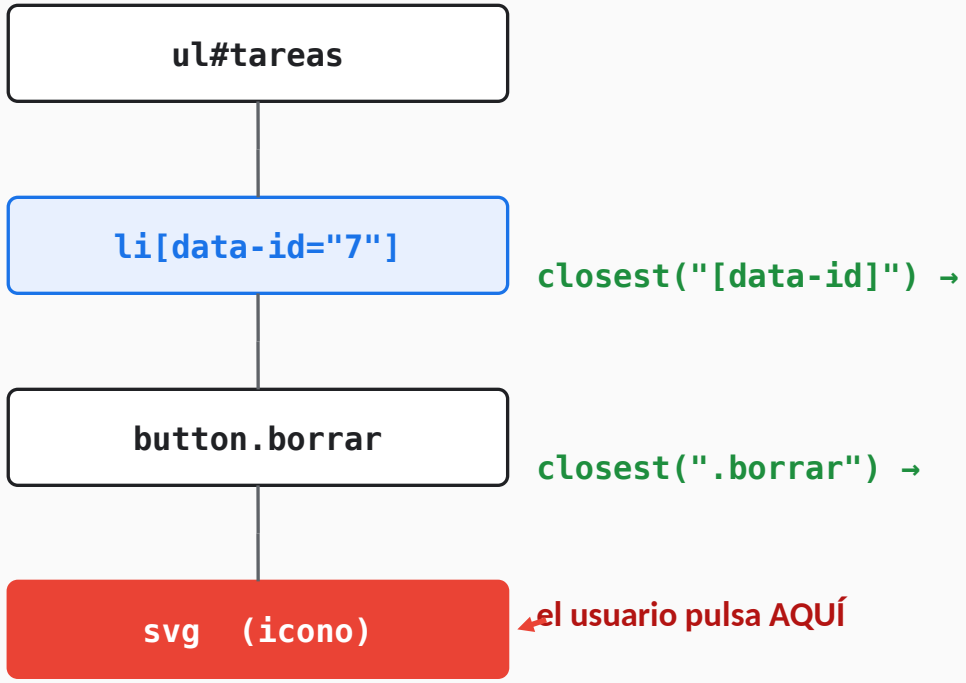
consola

```
li.dataset.id;           // "42"
Number(li.dataset.id);   // 42
li.dataset.hecha = "si";
// → data-hecha="si"
delete li.dataset.hecha;
```

**El patrón de toda la unidad:** al pintar, guardo el id del objeto en data-id; al hacer clic, leo dataset.id y busco el objeto en el modelo.

**CLAVE —** Compara con === después de convertir: dataset.id === 42 es false porque "42" es un string. Number(dataset.id) === 42.

# ¿Este elemento cumple el selector? ¿Cuál de sus padres lo cumple?




consola

```
const el = e.target;          // el svg
el.matches(".borrar");        // false
el.closest(".borrar");        // <button class="borrar">
el.closest("[data-id]");      // <li data-id="7">
el.closest("[data-id]").dataset.id; // "7"

el.closest("form");           // null (no hay form)
```

**Por qué importa:** el clic puede caer en el icono, en el texto o en el botón. closest encuentra el botón esté donde esté el clic.

**NOTA —** matches mira solo el propio elemento; closest mira el propio elemento y luego sube. Los dos usan selectores CSS.



BLOQUE 3

# Modificar y crear

Texto, clases y estilos, crear e insertar nodos, innerHTML y XSS, template y pintar desde el modelo

# El aspecto lo decide el CSS; JavaScript solo cambia la clase

| classList. | Hace | Ejemplo |
|---|---|---|
| `add("a", "b")` | Añade una o varias | li.classList.add("hecha") |
| `remove("a")` | Quita | li.classList.remove("hecha") |
| `toggle("a")` | Pone o quita (devuelve el estado) | li.classList.toggle("hecha") |
| `toggle("a", cond)` | Fuerza según la condición | li.classList.toggle("hecha", t.hecha) |
| `contains("a")` | ¿La tiene? | li.classList.contains("hecha") |
| `replace("a", "b")` | Cambia una por otra | btn.classList.replace("off", "on") |

**toggle con condición** es la forma limpia de reflejar el estado del modelo: no hace falta un if/else con add y remove. Y className = "..." BORRA todas las clases que había.

estilos.css

```css
.tarea.hecha {
  text-decoration: line-through;
  opacity: 0.6;
}
.tarea.vencida {
  border-left: 4px solid var(--rojo);
}
```

render.js

```js
li.textContent = tarea.titulo;
li.classList.toggle("hecha",
  tarea.hecha);
li.classList.toggle("vencida",
  tarea.estaVencida());
```

**CLAVE —** Separar responsabilidades: el CSS sabe cómo se ve «hecha»; el JavaScript solo sabe CUÁNDO una tarea está hecha.

# Cuándo sí tocar el estilo directamente

style.js

```
caja.style.backgroundColor = "gold"; // camelCase
caja.style.width = `${porcentaje}%`;  // ¡con unidad!
caja.style.width = 50;                // ignorado
caja.style.removeProperty("width");

// variables CSS (custom properties)
barra.style.setProperty("--progreso", "75%");

// el estilo FINAL, venga de donde venga
getComputedStyle(caja).display;  // "flex"
```

| Situación | Mejor |
|---|---|
| Estados (activo, error, hecha) | classList |
| Valor calculado (posición, % de progreso) | style o --variable |
| Temas (claro/oscuro) | clase en <html> + variables CSS |
| Saber cómo se ve de verdad | getComputedStyle |

**el.style solo LEE lo que puso style,** no lo de las hojas CSS.

**Truco limpio:** JavaScript pone el número en una variable (--progreso) y el CSS decide qué hacer con ella: width: var(--progreso). Así el diseño sigue en el CSS.

**NOTA —** style.width = 50 no da error: se ignora en silencio porque falta la unidad. Si algo «no hace nada», mira la pestaña Styles.

BLOQUE 3 · CREAR NODOS

# createElement → configurar → append

crear.js

```
function crearItem(tarea) {
  const li = document.createElement("li");
  li.className = "tarea";
  li.dataset.id = tarea.id;

  const texto = document.createElement("span");
  texto.textContent = tarea.titulo;

  const borrar = document.createElement("button");
  borrar.type = "button";
  borrar.textContent = "Borrar";
  borrar.dataset.accion = "borrar";

  li.append(texto, borrar);
  return li;
}
```

**1 · createElement** existe en memoria, NO en la página

↓

**2 · configurar** textContent, clases, dataset

↓

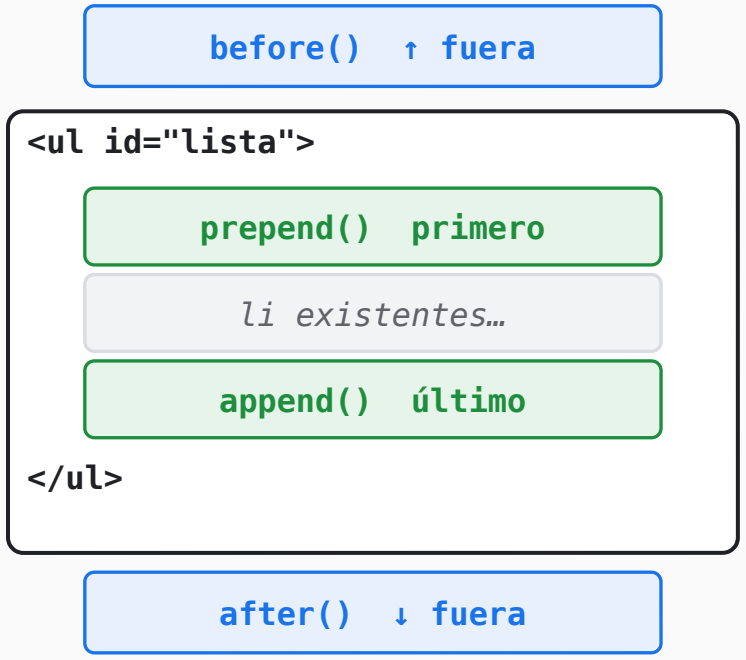
**3 · append** ahora sí se ve

```
lista.append(crearItem(tarea));
```

**append** admite varios nodos y strings; appendChild, uno.

**CLAVE —** Una función que recibe un objeto del modelo y devuelve un nodo. Esa función es vuestro «componente» antes de React.

# Dónde va el nodo nuevo… y qué pasa si ya estaba




dom.js

```
lista.prepend(nuevo);          // primer hijo
lista.append(nuevo);           // último hijo
item.before(nuevo); item.after(nuevo); // hermanos
item.replaceWith(nuevo);       // lo sustituye
item.remove();                 // fuera del árbol
lista.replaceChildren();       // vacía la lista
lista.replaceChildren(...nodos); // vacía y pone estos
```

**Un nodo solo está en un sitio:** append de un nodo que ya está en la página lo MUEVE, no lo copia. Para duplicar: nodo.cloneNode(true) (true = con sus hijos; los listeners no se copian).

**CLAVE —** Mover una tarea de «Por hacer» a «En curso» es literalmente columnaEnCurso.append(li). El mismo nodo, otro padre.

# Nunca metas datos del usuario con innerHTML

peligro.js

```
const titulo = input.value;  // lo escribe el usuario

// ✗ el navegador INTERPRETA lo que haya escrito
li.innerHTML = `<span>${titulo}</span>`;
```

seguro.js

```
// ✓ textContent no interpreta nada
const span = document.createElement("span");
span.textContent = titulo;
li.append(span);
```

**Si escribe:** `<img src=x onerror="alert(document.cookie)">`

**...su código se ejecuta en la página de todos los que vean esa tarea.**

| | ¿Datos del usuario? |
|---|---|
| textContent / createElement | ✓ Siempre seguro |
| innerHTML con texto FIJO tuyo | ✓ Aceptable |
| innerHTML con ${datos} | ✗ XSS |

**CLAVE —** XSS (Cross-Site Scripting): inyectar código en una web ajena a través de sus propios datos. Sale en todas las ediciones del OWASP Top 10 (desde 2021, dentro de «Injection»).

# El HTML de la tarjeta en el HTML, los datos con JavaScript

index.html

```html
<template id="tpl-tarea">
  <li class="tarea">
    <span class="tarea__titulo"></span>
    <time class="tarea__limite"></time>
    <button type="button"
      data-accion="borrar">Borrar</button>
  </li>
</template>
```

render.js

```js
const tpl = document.querySelector("#tpl-tarea");

function crearItem(tarea) {
  const li = tpl.content.firstElementChild
    .cloneNode(true);
  li.dataset.id = tarea.id;
  li.querySelector(".tarea__titulo")
    .textContent = tarea.titulo;
  return li;
}
```

### Inerte

no se muestra ni se ejecuta

### `cloneNode(true)`

una copia nueva cada vez

### `textContent`

rellenar sin riesgo de XSS

**NOTA —** Con template el maquetador puede cambiar la tarjeta sin tocar JavaScript, y vosotros no escribís createElement veinte veces.

# Una sola dirección: el modelo manda y el DOM obedece




main.js

```
function render() {
  lista.replaceChildren(...tablero.tareas.map(crearItem));
  contador.textContent = `${tablero.pendientes} pendientes`;
}
```

**La regla:** nunca cambies el DOM «a mano» en el manejador. Cambia el modelo y llama a render(). Así lo que se ve siempre coincide con los datos.

**CLAVE —** Esto es exactamente lo que hace React (UD7): estado → render. Hoy lo hacéis vosotros, y entenderéis qué os ahorra.

# Tocar el DOM es caro: agrupa los cambios




lento-rapido.js

```
// ✗ 1000 inserciones = 1000 cambios en la página
for (const t of tareas) lista.append(crearItem(t));

// ✓ un fragmento en memoria y UNA inserción
const frag = document.createDocumentFragment();
for (const t of tareas) frag.append(crearItem(t));
lista.append(frag);
```

## Reglas prácticas

- Construye fuera (fragmento o array de nodos) e inserta una vez
- replaceChildren(...nodos) ya inserta todo de golpe
- No leas medidas (offsetHeight, getBoundingClientRect) dentro de un bucle que escribe: fuerza un reflow en cada vuelta
- Para animar, clases con transition/transform (DIW), no style.top en un bucle
- Con 50 tareas no lo notaréis; con 5000, sí

**NOTA —** El navegador ya agrupa muchos cambios seguidos, pero un DocumentFragment o replaceChildren lo deja claro y evita sorpresas.



BLOQUE 4

# Eventos

addEventListener, el objeto evento, propagación, delegación, listeners duplicados y this

# Elemento + tipo de evento + función que reacciona

```
boton.addEventListener("click", alPulsar);
```

- `boton` — ¿QUIÉN?
- `"click"` — ¿QUÉ PASA?
- `alPulsar` — ¿QUÉ HAGO?

eventos.js

```
const boton = document.querySelector("#nueva");

function alPulsar(evento) {
  console.log("pulsado", evento.type);
}
boton.addEventListener("click", alPulsar);

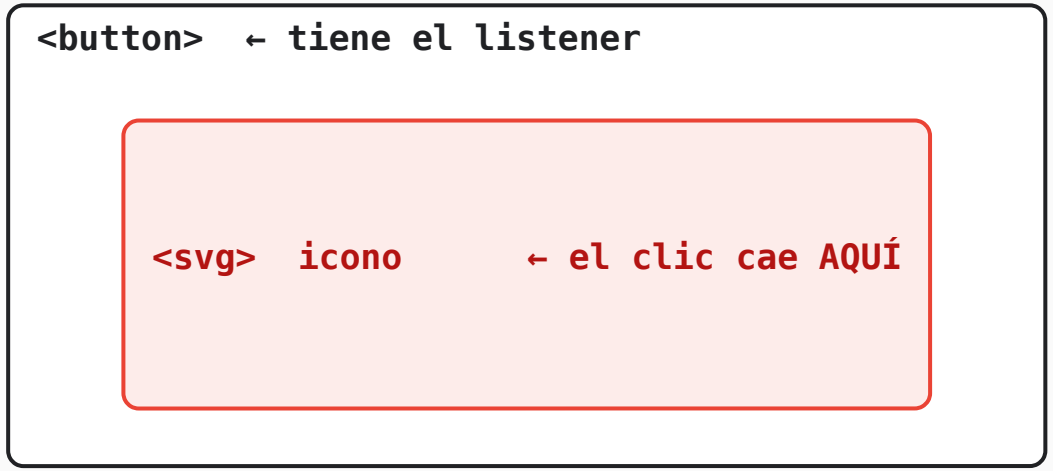
// ✗ alPulsar() la EJECUTA ahora y pasa undefined
boton.addEventListener("click", alPulsar());
```

| Forma | ¿Varios a la vez? | ¿Se usa? |
|---|---|---|
| `<button onclick="…">` | ✗ | No: mezcla HTML y JS |
| `boton.onclick = f` | ✗ (el 2.º pisa al 1.º) | Poco |
| `addEventListener` | ✓ | Sí, siempre |

**Se pasa la función, no su resultado:** alPulsar sin paréntesis. El navegador la llamará cuando ocurra el evento y le dará el objeto evento.

**CLAVE —** Es un callback como los de map o forEach (UD3): le das una función y otro decide cuándo llamarla.

# target es dónde ocurrió; currentTarget, quién escucha




| Propiedad | Valor en este clic |
|---|---|
| `e.target` | el <svg> (lo más profundo) |
| `e.currentTarget` | el <button> (quien escucha) |
| `e.type` | "click" |
| `e.timeStamp` | ms desde que cargó la página |

evento.js

```
boton.addEventListener("click", (e) => {
  e.target;          // <svg> … o <span>, o el botón
  e.currentTarget;   // siempre el botón
  e.target === e.currentTarget; // depende de dónde
});

campo.addEventListener("keydown", (e) => {
  e.key;     // "Enter", "a", "Escape", "ArrowUp"
  e.shiftKey; // ¿con Mayús?
});

zona.addEventListener("pointermove", (e) => {
  e.clientX; e.clientY; // posición
});
```

**CLAVE —** Si usas e.target y luego te sale undefined al leer dataset, es que el clic cayó en un hijo. Solución: e.currentTarget o closest().

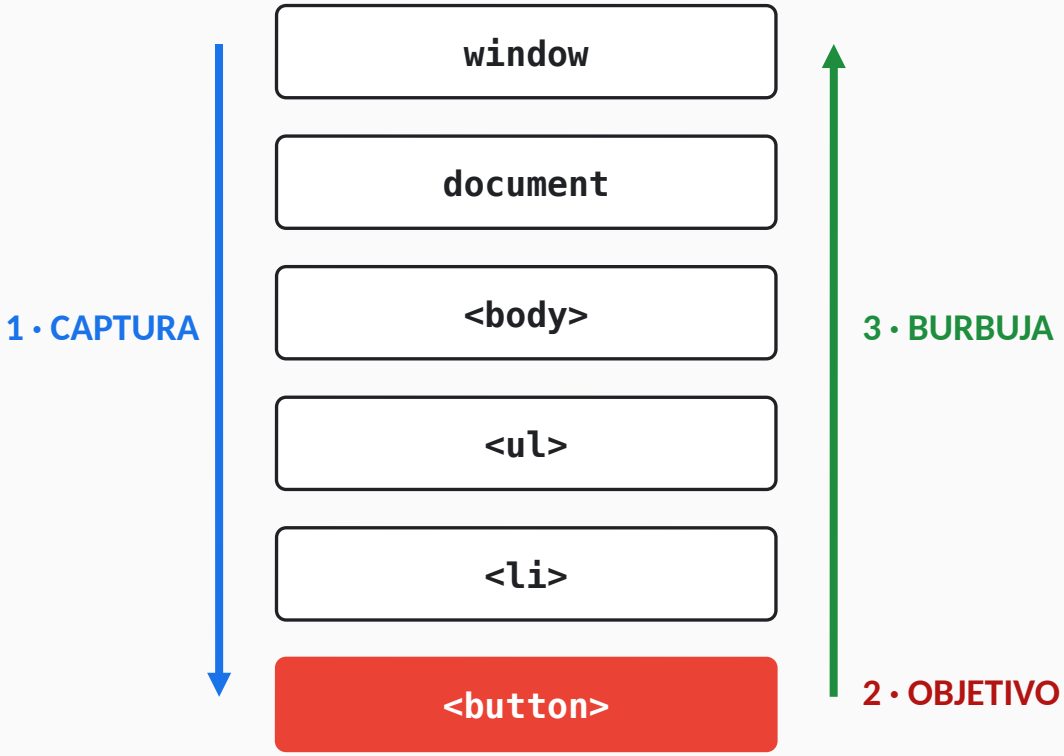
# Los que vais a usar el 90 % de las veces

| Evento | Cuándo salta | Típico para |
|---|---|---|
| `click` | Pulsar (ratón, dedo o Enter/espacio en un botón) | Botones, enlaces |
| `input` | Cada cambio en un campo, al momento | Buscar mientras escribe, contador |
| `change` | Al confirmar: soltar el select, marcar, salir del campo | select, checkbox, radio |
| `submit` | Enviar el formulario (botón o Enter) | Alta de datos (bloque 5) |
| `keydown` | Pulsar una tecla (e.key) | Atajos, Escape para cerrar |
| `focusin / focusout` | Entrar o salir de un campo (sí burbujean) | Validar al salir |
| `pointerenter / leave` | El puntero entra o sale (no burbujean) | Tooltips |
| `DOMContentLoaded` | Árbol listo (bloque 1) | Arrancar la app |
| `scroll / resize` | Desplazar / cambiar tamaño (¡cientos por segundo!) | Con cuidado |

**NOTA —** click también salta con teclado en <button> y en <a href>. Por eso los botones de verdad son mejores que un  con click (bloque 4, accesibilidad).

BLOQUE 4 · PROPAGACIÓN

# Un clic viaja: baja (captura), llega (objetivo) y sube (burbuja)




consola

```
ul.addEventListener("click",
  () => console.log("ul"));
li.addEventListener("click",
  () => console.log("li"));

// clic en el botón:
// li
// ul
```

**Por defecto se escucha en la burbuja.**
addEventListener(tipo, f, { capture: true }) escucha al bajar. Casi nunca hace falta.

**CLAVE —** Gracias a la burbuja, un listener en el <ul> se entera de los clics en TODOS sus <li>. Eso es la delegación (dentro de dos diapositivas).

BLOQUE 4 · PARAR EL EVENTO

# stopPropagation y preventDefault hacen cosas distintas

### e.preventDefault()

Cancela lo que el NAVEGADOR haría por su cuenta: enviar el formulario y recargar, seguir el enlace, marcar la casilla. El evento sigue subiendo.

### e.stopPropagation()

Para el VIAJE: los padres no se enteran. El navegador sí hace su acción por defecto. Úsalo poco: rompe la delegación y la analítica.

prevent.js

```
form.addEventListener("submit", (e) => {
  e.preventDefault();   // ¡que no recargue!
  guardar(new FormData(form));
});

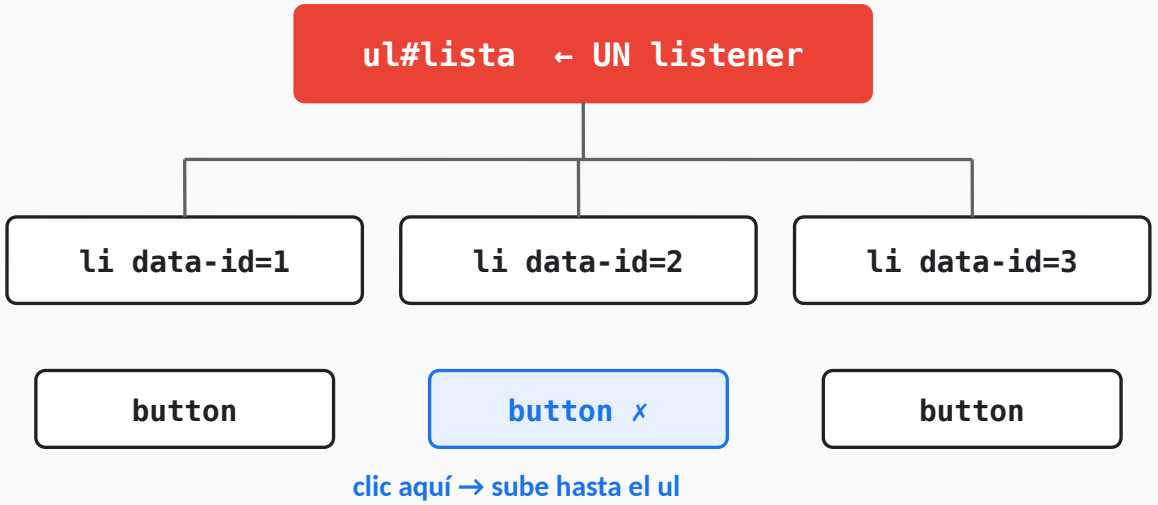
enlace.addEventListener("click",
  (e) => e.preventDefault());
```

stop.js

```
// modal: clic fuera cierra, clic dentro no
fondo.addEventListener("click", cerrar);
caja.addEventListener("click", (e) => {
  e.stopPropagation(); // no llega al fondo
});
// mejor aún: if (e.target === fondo) cerrar();
```

**CLAVE —** El fallo n.º 1 de los formularios: olvidar preventDefault en submit. La página se recarga y parece que «no hace nada».

# Un solo listener en el padre para todos los hijos, también los futuros




**Tres ventajas:** un listener en vez de mil, sirve para los elementos que pintes después y no se duplica al repintar.

delegacion.js

```
lista.addEventListener("click", (e) => {
  const boton = e.target.closest("[data-accion]");
  if (!boton) return;           // clic en otro sitio

  const id = Number(boton.closest("li").dataset.id);
  const accion = boton.dataset.accion;

  if (accion === "borrar") tablero.borrar(id);
  if (accion === "hecha") tablero.completar(id);
  render();
});
```

**CLAVE —** El patrón completo: escuchar en el padre → closest para saber qué se pulsó → dataset para saber qué objeto → modelo → render.



BLOQUE 4 · LISTENERS DUPLICADOS

# Si añades el listener dentro de render(), cada repintado suma otro

mal.js

```
function render() {
  lista.replaceChildren(...items);
  // ✗ cada render añade OTRA flecha nueva
  nuevaBtn.addEventListener("click", () => crear());
}
// 3 renders → 1 clic crea 3 tareas
// (con la MISMA función, crear, no se duplicaría)
```

## Arreglos

- Los listeners se ponen UNA vez, al arrancar, fuera de render()
- Para los hijos que se repintan: delegación en el padre
- Los nodos que borras se llevan sus listeners: no hace falta quitarlos

quitar.js

```
// quitar: hace falta LA MISMA función
boton.addEventListener("click", f);
boton.removeEventListener("click", f); // ✓

boton.addEventListener("click", () => f());
boton.removeEventListener("click", () => f()); // ✗

// solo una vez
boton.addEventListener("click", f, { once: true });

// quitar varios de golpe
const ctrl = new AbortController();
document.addEventListener("keydown", f,
  { signal: ctrl.signal });
ctrl.abort();
```



**NOTA —** La misma función añadida dos veces se ignora; una flecha nueva es otra función (=== da false, como dos objetos en la UD4). Por eso se duplica y removeEventListener no la encuentra.

# El this perdido de la UD4, otra vez

| Manejador | this dentro |
|---|---|
| `function () { … }` | el elemento (currentTarget) |
| `() => { … }` | el this de fuera |
| `obj.metodo  (sin llamar)` | el elemento: ¡se pierde obj! |
| `() => obj.metodo()` | obj ✓ |
| `obj.metodo.bind(obj)` | obj ✓ |

**Con #privados el error se nota:** TypeError: Cannot read private member #valor from an object whose class did not declare it. Significa: this no es un Contador.

contador.js

```
class Contador {
  #valor = 0;
  constructor(boton, salida) {
    this.salida = salida;
    // ✗ boton.addEventListener("click", this.sumar);
    //   → this = el botón: this.#valor revienta
    boton.addEventListener("click", () => this.sumar());
  }
  sumar() {
    this.#valor++;
    this.salida.textContent = this.#valor;
  }
}
```

**CLAVE —** Dentro de clases, la flecha envolvente () => this.metodo() es la solución más legible. Es el mismo arreglo del setInterval de la UD4.

# Si se puede pulsar, que sea un <button>

### `<div class="btn"> + click` ✗

No recibe el foco con Tab, no reacciona a Enter ni a espacio y el lector de pantalla no dice «botón». Hay que añadir a mano tabindex, role y keydown.

### `<button type="button"> + click` ✓

Foco, Enter, espacio y el rol «botón» gratis. type="button" para que dentro de un formulario no lo envíe.

teclado.js

```
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") cerrarModal();
  if (e.key === "n" && e.ctrlKey) {
    e.preventDefault(); nuevaTarea();
  }
});
```

### Comprobadlo así

- Desenchufa el ratón: ¿puedes usar toda la app con Tab, Enter y Escape?
- ¿Se ve dónde está el foco? (:focus-visible en el CSS)
- Al abrir algo, mueve el foco con elemento.focus()

**NOTA —** e.key da la tecla con su significado ("a", "A", "Enter"); e.code, la posición física ("KeyA"). Para atajos, casi siempre e.key.

# Tu propio evento, con datos dentro

emitir.js

```
// quien avisa
const ev = new CustomEvent("tarea-completada", {
  detail: { id: 7, titulo: "Entregar" },
  bubbles: true,
});
item.dispatchEvent(ev);
```

escuchar.js

```
// quien escucha, sin conocer a quien avisa
document.addEventListener(
  "tarea-completada", (e) => {
    console.log(e.detail.titulo);
    mostrarAviso(`¡Hecho: ${e.detail.titulo}!`);
  });
```

**Para qué:** desacoplar. La lista no necesita saber que existe un contador o un aviso: lanza su evento y quien quiera lo escucha. Así funcionan los Web Components.

**NOTA —** Es opcional en esta unidad, pero lo veréis en librerías. bubbles: true para que suba como un click.



BLOQUE 5

# Formularios

Leer datos con FormData, input/change/submit, validación nativa, Constraint Validation y errores accesibles

# submit + preventDefault + FormData

index.html

```html
<form id="alta">
  <input name="titulo" required>
  <select name="prioridad">
    <option>baja</option>
    <option selected>media</option>
  </select>
  <input name="horas" type="number">
  <button>Añadir</button>
</form>
```

alta.js

```js
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const datos = Object.fromEntries(new FormData(form));
  // { titulo: "…", prioridad: "media", horas: "3" }
  datos.horas = Number(datos.horas);
  tablero.crearTarea(datos);
  form.reset();
  render();
});
```

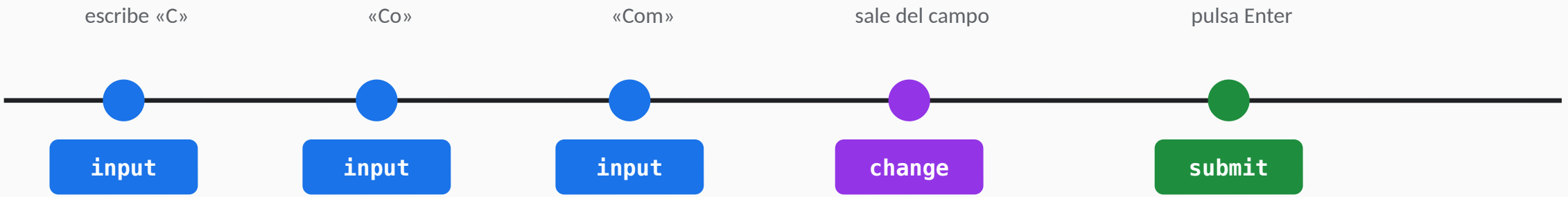
**name** — FormData usa el atributo name; sin name, el campo no sale

**String** — todo llega como string: "3", no 3. Number() para los números

**reset()** — deja el formulario como estaba al cargar

**CLAVE —** Escuchad submit en el <form>, no click en el botón: así funciona también al pulsar Enter dentro de un campo.

# Cada uno en su momento




buscar.js

```
// filtrar mientras escribe
buscador.addEventListener("input", () => {
  filtro = buscador.value.trim().toLowerCase();
  render();
});

orden.addEventListener("change", render);
```

| Evento | Úsalo para |
|---|---|
| `input` | Filtros en vivo, contadores de caracteres |
| `change` | select, checkbox, radio, y validar al salir |
| `submit` | Guardar: SIEMPRE con preventDefault |

**NOTA —** En checkbox y radio, change salta al marcar (no hace falta salir). Lee .checked, no .value.

# El HTML ya sabe validar: úsalo primero

| Atributo | Comprueba | Ejemplo |
|---|---|---|
| `required` | Que no esté vacío | <input required> |
| `type="email" / "url"` | Formato | <input type="email"> |
| `minlength / maxlength` | Longitud del texto | minlength="3" |
| `min / max / step` | Rango numérico o de fecha | type="number" min="1" |
| `pattern` | Expresión regular (todo el valor) | pattern="[0-9]{5}" |

estilos.css

```css
/* solo tras tocar el campo */
input:user-invalid {
  border-color: var(--rojo);
}
input:user-valid {
  border-color: var(--verde);
}
```

**novalidate en el <form>** desactiva los globos del navegador para mostrar tus mensajes, pero los atributos siguen funcionando para la Constraint Validation API (siguiente diapositiva).

**Sin JavaScript:** el navegador no deja enviar el formulario si algo falla y muestra su propio mensaje.

**NOTA —** :invalid marca el campo en rojo desde que carga la página; :user-invalid solo cuando el usuario ya lo ha tocado. Mejor experiencia.

# Preguntarle al navegador qué falla en cada campo

| input.validity. | true si... |
|---|---|
| `valueMissing` | required y vacío |
| `typeMismatch` | no parece un email / URL |
| `tooShort / tooLong` | minlength / maxlength (*) |
| `rangeUnderflow / Overflow` | menor que min / mayor que max |
| `patternMismatch` | no cumple pattern |
| `customError` | pusiste setCustomValidity |
| `valid` | todo bien |

**setCustomValidity("")** quita el error. Si no la llamas, el campo se queda inválido para siempre.

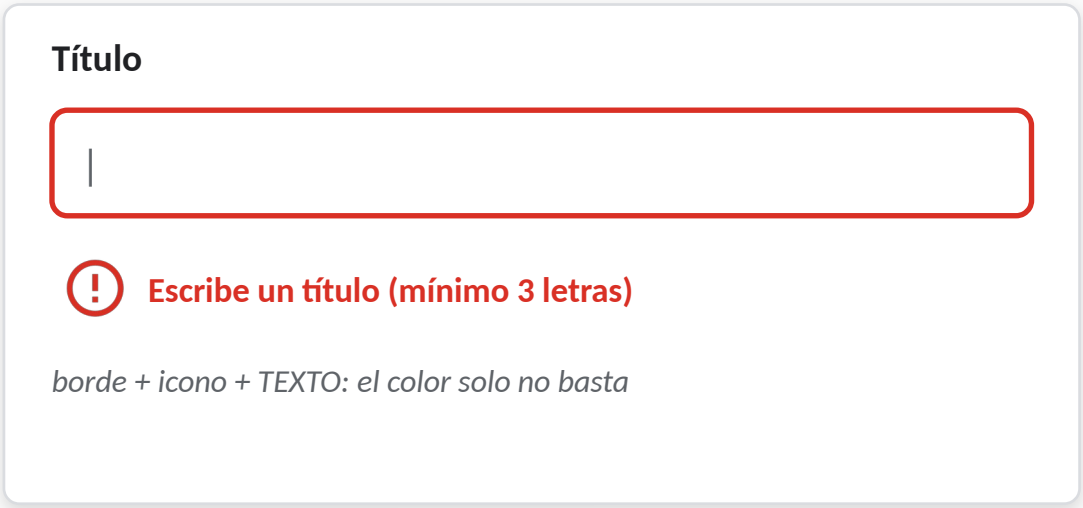
validar.js

```
function validarTitulo() {
  const t = form.elements.titulo;
  if (t.validity.valueMissing) {
    t.setCustomValidity("Escribe un título");
  } else if (tablero.existe(t.value.trim())) {
    t.setCustomValidity("Ya hay una tarea así");
  } else {
    t.setCustomValidity(""); // ← ¡imprescindible!
  }
  return t.checkValidity();
}

form.elements.titulo.addEventListener("input", validarTitulo);
```

**NOTA —** (*) Solo con lo que ESCRIBE el usuario: si pones el valor con JavaScript, tooShort sigue en false. form.checkValidity() mira todos los campos.

# Junto al campo, en texto, y que el lector de pantalla lo diga




index.html

```html
<label for="titulo">Título</label>
<input id="titulo" name="titulo" required
  minlength="3" aria-describedby="err-titulo">
<p id="err-titulo" class="error" aria-live="polite"></p>
```

errores.js

```js
function mostrarError(campo, mensaje) {
  campo.setAttribute("aria-invalid", mensaje ? "true" : "false");
  document.querySelector(`#err-${campo.name}`).textContent = mensaje;
}
```

**CLAVE —** aria-describedby une el campo con su mensaje; aria-live="polite" hace que se lea al cambiar. Al enviar con errores, pon el foco en el primero.

# Es comodidad para el usuario, no una barrera

### Usuario normal

ve el error al momento: mejor experiencia

### Atacante

abre DevTools, quita required, o manda la petición con curl

### Servidor

vuelve a validar TODO: es el único que manda

**Regla:** valida en el cliente para ayudar, y en el servidor para proteger. En la UD6 enviaréis datos con fetch: el servidor tiene que comprobarlos otra vez. Y nunca confiéis en un precio, un rol o un id que venga del navegador.

**NOTA —** Lo mismo vale para hidden, disabled o un botón oculto: todo lo que está en el navegador lo puede cambiar el usuario.



BLOQUE 6

# Manos a la obra

Laboratorio, DevTools, caso real, errores comunes, control y mini-reto

LABORATORIO

# Agenda de contactos con DOM y eventos

1



**Modelo**

Contacto y Agenda (clases, UD4) en src/modelo/

2



**Pintar**

<template> + render() desde la agenda

3



**Alta**

formulario con validación y errores en el campo

4



**Acciones**

favorito y borrar con UN listener (delegación)

5



**Filtro**

buscar mientras escribe + contador

Proyecto Vite en Docker (como la UD4). Toda la lógica en el modelo; main.js solo conecta eventos → modelo → render(). Nada de innerHTML con datos del usuario.

**ÉXITO —** Criterio de éxito: alta validada sin recargar, borrar y favorito con un único listener en la lista, filtro en vivo y ningún listener duplicado tras 10 altas.

# Quién hace qué

| | |
|---|---|
| `index.html` | form#alta · input#buscar · ul#contactos · template#tpl-contacto |
| `src/modelo/Contacto.js` | nombre, email (validados) · favorito · alternarFavorito() |
| `src/modelo/Agenda.js` | #contactos · añadir · borrar · buscar(texto) · get total |
| `src/vista/render.js` | crearItem(contacto) con el template · render(lista, contactos) |
| `src/main.js` | listeners (una vez) → agenda → render() |

**Si acabáis:** ordenar (favoritos primero) con un <select> y change, y Escape para limpiar el buscador.

main.js

```
form.addEventListener("submit", alta);
buscar.addEventListener("input", pintar);
lista.addEventListener("click", (e) => {
  const b =
    e.target.closest("[data-accion]");
  if (!b) return;
  const li = b.closest("li");
  const id = Number(li.dataset.id);
  if (b.dataset.accion === "borrar")
    agenda.borrar(id);
  else agenda.alternarFavorito(id);
  pintar();
});
pintar();
```

**NOTA —** El laboratorio es la base del calentamiento B de la Práctica 2 (06/11): el mismo ciclo, con el tablero de tareas.

DEVTOOLS

# El DOM se inspecciona en vivo

| | |
|---|---|
| `Elements` | Ver el árbol REAL (no el HTML original). Editar a mano para probar |
| `$0` | En la consola: el último elemento seleccionado en Elements |
| `Event Listeners` | Pestaña dentro de Elements: qué escucha cada nodo y en qué línea |
| `Break on → subtree modifications` | Pausa cuando algo cambia ese nodo: ¿quién me borra la lista? |
| `getEventListeners($0)` | Solo en la consola de Chrome: los listeners del elemento |
| `monitorEvents($0, "click")` | Imprime cada evento que recibe |

```
> getEventListeners($0)
▼ {click: Array(1)}
  ▼ click: Array(1)
    ▼ 0:
        listener: ƒ (e)
        once: false
        passive: false
        type: "click"

> $0.dataset
  DOMStringMap {id: '7'}
```

**NOTA —** Si en Event Listeners salen 3 listeners de click en el mismo botón, ahí tienes el bug de la diapositiva 34.

CASO REAL

# Dónde os vais a encontrar todo esto

### React, Vue, Svelte

Por debajo llaman a createElement, append y addEventListener. React agrupa los eventos con delegación en la raíz.

### Web Components

Etiquetas propias (<mi-tablero>) con su DOM, sus eventos y CustomEvent.

### Analítica

Google Analytics y similares escuchan clics en document gracias a la burbuja.

### Seguridad

Content-Security-Policy y la API Trusted Types existen para frenar el XSS de innerHTML.

### Accesibilidad

Desde junio de 2025, la Ley Europea de Accesibilidad obliga a muchos servicios digitales a cumplir la norma EN 301 549.

### Extensiones

Un bloqueador de anuncios es querySelectorAll + remove sobre páginas ajenas.

**DATO —** Probadlo: en cualquier web, `$$("a").length` en la consola cuenta sus enlaces (`$$` = querySelectorAll en DevTools).

ERRORES COMUNES (1/2)

# Seleccionar y modificar

## El elemento es null

El script corre antes que el HTML, o el selector está mal

```
document.querySelector("lista")
```

```
document.querySelector("#lista")
```

## innerHTML con datos

Abre la puerta al XSS

```
li.innerHTML = tarea.titulo
```

```
li.textContent = tarea.titulo
```

## map sobre una NodeList

NodeList no tiene map

```
document.querySelectorAll("li").map(…)
```

```
[...document.querySelectorAll("li")].map(…)
```

## dataset sin convertir

Siempre es string

```
if (li.dataset.id === 7)
```

```
if (Number(li.dataset.id) === 7)
```

ERRORES COMUNES (2/2)

# Eventos y formularios

## Llamar al manejador

Se ejecuta ya y se pasa undefined

```
btn.addEventListener("click", crear())
```

```
btn.addEventListener("click", crear)
```

## Formulario que recarga

Falta cancelar el envío

```
form.onsubmit = guardar
```

```
e.preventDefault(); guardar();
```

## Listener dentro de render()

Cada repintado añade otro

```
function render() { btn.addEventListener(…) }
```

```
btn.addEventListener(…) una vez, fuera
```

## setCustomValidity sin limpiar

El campo no vuelve a ser válido

```
if (mal) c.setCustomValidity("Error")
```

```
c.setCustomValidity(mal ? "Error" : "")
```

CONTROL DE COMPRENSIÓN

# Preguntas de control y debate

1. ¿Por qué ul.childNodes.length da 5 si solo hay dos <li>? ¿Qué usarías?
2. Un <input value="1"> en el que el usuario escribe 5: ¿qué dan getAttribute("value") y .value?
3. Clic en el <svg> de un botón con listener: ¿qué son e.target y e.currentTarget?
4. ¿Por qué la delegación funciona con los <li> que se añaden después?
5. li.innerHTML = `<b>${titulo}</b>` con el título que escribe el usuario: ¿qué riesgo hay y cómo lo arreglas?

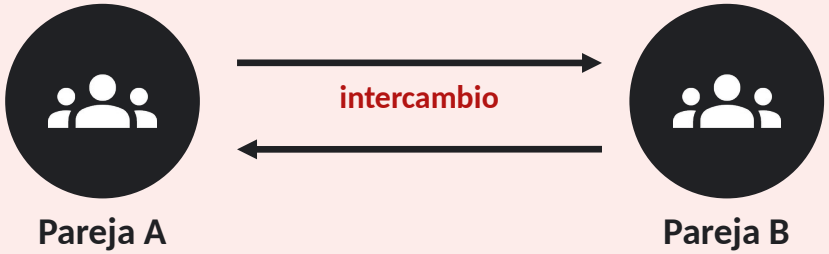
MINI-RETO PRÁCTICO

# Reto express: 20 minutos, en parejas

| | | |
|---|---|---|
| 1 | **Pestañas** | 3 botones con data-panel y 3 paneles |
| 2 | **Delegación** | un solo listener en la barra de pestañas |
| 3 | **Estado** | aria-selected y hidden según la pestaña activa |
| 4 | **Teclado** | flechas ← → cambian de pestaña |
| 5 | **Sin innerHTML** | solo classList, hidden y atributos |

### Revisión cruzada

A los 15 min, intercambiad el código y comprobad:




- ¿Hay un único addEventListener para las pestañas?
- ¿Se puede usar solo con teclado?
- ¿Solo un panel visible y aria-selected coherente?

**ÉXITO —** Criterio de éxito: un listener, el panel correcto visible, aria-selected="true" solo en la activa y las flechas funcionando.

GLOSARIO (1/2)

# DOM

| | |
|---|---|
| `DOM` | El documento como árbol de objetos |
| `nodo` | Cada pieza del árbol: elemento, texto… |
| `elemento` | Nodo que corresponde a una etiqueta |
| `document` | Raíz del DOM y punto de entrada |
| `window / BOM` | El navegador: location, history, storage… |
| `DOMContentLoaded` | El árbol ya está construido |
| `querySelector` | Primer elemento que cumple un selector |
| `NodeList` | Lista de nodos (estática en querySelectorAll) |
| `HTMLCollection` | Lista de elementos que se actualiza sola |
| `textContent` | Texto de un nodo; asignarlo no interpreta HTML |
| `dataset` | Acceso a los atributos data-* |
| `XSS` | Inyección de código a través de datos |

GLOSARIO (2/2)

# Eventos y formularios

| | |
|---|---|
| `evento` | Algo que ocurre: clic, tecla, envío… |
| `listener` | Función que reacciona a un evento |
| `target` | Elemento donde ocurrió el evento |
| `currentTarget` | Elemento cuyo listener se está ejecutando |
| `propagación` | Captura → objetivo → burbuja |
| `delegación` | Un listener en el padre para todos los hijos |
| `preventDefault` | Cancela la acción por defecto del navegador |
| `stopPropagation` | Detiene el viaje del evento |
| `FormData` | Los campos de un form por su name |
| `validity` | Estado de validación de un campo |
| `setCustomValidity` | Mensaje de error propio ("" lo quita) |
| `aria-live` | Región que el lector anuncia al cambiar |

ÚLTIMA REFLEXIÓN

# La interfaz es el modelo visto desde fuera

### Una sola dirección

Evento → modelo → render. Si el DOM y los datos no coinciden, el fallo está en saltarse ese orden.

### Desconfía de la entrada

Todo lo que escribe el usuario es texto: textContent para pintarlo, y el servidor vuelve a validar.

### Para todo el mundo

Botones de verdad, foco visible, errores en texto. No es un extra: es parte del trabajo.

**Pregunta para casa:** abre una web que uses a diario, inspecciona un botón y mira en Event Listeners quién lo escucha. ¿Está en el botón o en un padre?

CHECKPOINT

# Resumen de puntos clave

- La página es un árbol de objetos: el DOM
- querySelector + closest + dataset para encontrar elementos y datos
- textContent y createElement; innerHTML nunca con datos del usuario
- Evento → modelo → render(), los listeners una sola vez
- Delegación: un listener en el padre para todos los hijos
- submit + preventDefault + FormData + validity

***Hoy: que se vea y responda. Después: que recuerde y hable con el servidor.***

SIGUIENTE · UD6

## Asincronía y almacenamiento

- Event loop, promesas y async/await
- fetch contra una API
- Guardar el tablero en localStorage